

Functional Requirements Document

Project Title: **Open Your Window**

Team Name: **De Biles**

Illia Uriupin and Maksym Potushynskyi

Contents

- Introduction2
- Functional Requirements2
 - Environmental Data Collection.....2
 - Humidity Calculation3
 - Window Recommendation Algorithm.....3
 - Notifications and User Interface4
 - LED Feedback4
- Non-Functional Requirements4
 - Performance4
 - Reliability.....4
 - Usability5
 - Security5
 - Scalability.....5
- Technical Architecture.....5
 - Hardware Components.....5
 - Software Components6
- Success Criteria6
- Risk Analysis7
 - Battery Availability Risk.....7
 - Sensor Measurement Accuracy7
 - Network Dependency7

Introduction

Ireland has a high average annual relative humidity of about **83%**, which often leads to excessive indoor moisture in residential buildings. High humidity can cause mold growth, condensation, and moisture accumulation in walls and furniture. It also affects human comfort and health by reducing sweat evaporation and promoting the growth of microorganisms such as mold and bacteria. In technical environments like **server rooms**, moisture can even damage electronic equipment.

The simplest way to reduce indoor humidity is ventilation through opening a window. However, this is effective only when the outdoor air contains less water vapor than the indoor air, which is not always obvious in Ireland's humid and frequently changing climate.

For example, consider a **12 m² room with a ceiling height of 2.5 m ($\approx 30 \text{ m}^3$ of air)** at **22 °C**. At this temperature, saturated air contains about **19.4 g/m³** of water vapor. If the indoor relative humidity is **50%**, the absolute humidity is approximately **9.7 g/m³** (used formula for AH calculation is shown in Functional Requirements paragraph), meaning the room air contains about **291 g of water vapor** in total. A resting person typically releases around **60 g of water vapor per hour** through breathing and perspiration. In a closed room, this increases the humidity by roughly **2 g/m³** ($60/12 \text{ m}^2 / 2.5 \text{ m}$), which corresponds to about **8–10% relative humidity per hour**.

The **Open Your Window** project aims to address this problem by creating a system that monitors indoor conditions and compares them with outdoor weather data. Using a Raspberry Pi with a Sense HAT, the system calculates indoor and outdoor absolute humidity (AH) and recommends whether opening the window will reduce indoor moisture. The results are presented through a local web interface and optional notifications, helping users maintain a healthier indoor environment.

Functional Requirements

The system monitors indoor environmental conditions and compares them with outdoor weather data to determine whether opening a window will reduce indoor humidity. The system operates in four stages: environmental data collection, humidity calculation, decision making, and user notification.

Environmental Data Collection

Indoor environmental data is measured using the Sense HAT sensors.

The system collects:

- indoor temperature T_i
- indoor relative humidity RH_i

Measurements are taken periodically with a configurable interval of 30–120 seconds.

Outdoor weather data is retrieved through a weather API (e.g., Open-Meteo) and includes:

- outdoor temperature T_o
- outdoor relative humidity RH_o

These values allow the system to compare indoor and outdoor moisture conditions.

Humidity Calculation

Since relative humidity depends on temperature, the system converts RH to absolute humidity (AH), which represents the actual mass of water vapor in air.

$$AH = \frac{6.112 \times e^{\left(\frac{17.67T}{T+243.5}\right)} \times RH \times 2.1674}{273.15 + T}$$

The system calculates indoor AH_i and outdoor AH_o .

The difference between these values determines whether ventilation will improve indoor conditions.

$$\Delta AH = AH_o - AH_i$$

Window Recommendation Algorithm

Window recommendations are determined using user-defined thresholds:

- maxRH — maximum indoor relative humidity
- minT — minimum indoor temperature
- mode — decision priority (humidity, temperature, or balanced)

Example opening condition (if mode is in balanced priority):

$$\text{if } (AH_i > \text{maxAH}) \text{ AND } (AH_i - AH_o \geq 5) \text{ AND } (T_i > \text{minT}) \\ \rightarrow \text{recommend opening window}$$

The system recommends closing the window if the room becomes too cold, ventilation is ineffective, or the target humidity level has been reached.

Notifications and User Interface

The system provides a local web interface (e.g., <http://raspberrypi.local>) displaying indoor and outdoor temperature, relative humidity, absolute humidity, and the recommended window state. Users can configure humidity and temperature thresholds as well as notification preferences.

Notifications are delivered through `ntfy.sh` and inform the user about ventilation recommendations, threshold violations, or system errors.

LED Feedback

The Sense HAT LED matrix provides simple visual indicators:

- Green — ventilation recommended
- Blue — humidity within acceptable range
- Red — humidity too high

Non-Functional Requirements

Performance

The system processes environmental data locally on the Raspberry Pi. Calculations such as humidity conversion and threshold evaluation require minimal computational resources and are expected to be completed within milliseconds.

Sensor measurements and weather API data retrieval occur periodically, every 30–120 seconds. The web interface updates automatically whenever new data becomes available.

Because the system runs locally, response times for the user interface remain very low and independent of external server load.

Reliability

Reliability is essential because the system is intended for continuous environmental monitoring.

All critical operations must include error handling. Possible failures include:

- network connectivity issues
- weather API request failures

Errors are logged and displayed on the local web interface. Additionally, the system sends error notifications to the user using ntfy.sh.

To improve system availability, the project will use a UPS module for the Raspberry Pi, allowing the system to remain operational during temporary power outages.

Usability

The system is designed to be simple to deploy and easy to use. The intended user experience is “plug-and-play”.

After connecting the Raspberry Pi to power and Ethernet, the device automatically hosts a local web page where users will be able to set environmental thresholds and configurations in message system.

Users do not need advanced technical knowledge to operate the system.

Security

Because the system primarily operates within a local network, the attack surface is minimal.

Scalability

The system will be designed to be modular and extensible.

Future extensions may include any user ideas such as integration with automated window openers or dehumidifiers.

The codebase will be open-source and publicly available on GitHub, enabling further development by the community.

Technical Architecture

Hardware Components

Main hardware elements:

- Raspberry Pi 3 Model B+

- Sense HAT for environmental sensing
- Geekworm GPIO Extension Header (G341) / 1-to-2 GPIO adapter
- Waveshare UPS HAT (E) power backup module
- 4× Li-ion batteries (21700 form factor)

The Raspberry Pi performs all calculations, runs the local web server, and communicates with external weather APIs.

The Sense HAT provides temperature and humidity measurements as well as an LED display.

The UPS module ensures continued operation during temporary power outages.

Software Components

The system software is implemented primarily in Python 3.

Key software modules include:

- Python runtime environment
- sense hat library for sensor communication
- requests module for weather API access
- local web server framework
- notification integration with ntfy.sh

Success Criteria

The project will be considered successful if the system meets the following measurable objectives:

- The system continuously monitors indoor environmental conditions without interruption for extended periods.
- The window recommendation algorithm correctly identifies situations where ventilation reduces indoor humidity.
- The local web interface reliably displays real-time environmental data.
- Notifications are delivered successfully when configured thresholds are exceeded.

- Users can configure thresholds and notification settings without modifying the source code.
- Indoor humidity levels can be maintained within a comfortable range (typically 40–60% RH) through the system’s recommendations.

Additionally, the project will be considered successful if the software is published on GitHub and can be easily deployed by other users.

Risk Analysis

Several risks may affect the successful deployment of the system.

Battery Availability Risk

The project uses a Waveshare UPS HAT (E) module that requires four Li-ion batteries in the 21700 form factor.

Although the UPS module has already been purchased and delivered, the main risk is the availability of compatible batteries within the required timeframe.

Mitigation strategy is to allow the system to operate without UPS support (what won’t be a problem at all).

Sensor Measurement Accuracy

During testing it was discovered that the Sense HAT temperature sensor is located close to the Raspberry Pi CPU, which can produce inaccurate readings due to heat generated by the processor and surrounding components.

To mitigate this issue, the project uses a Geekworm GPIO Extension Header (G341) / 1-to-2 GPIO adapter. This allows the Sense HAT to be physically separated from the Raspberry Pi, reducing thermal interference and improving measurement accuracy.

Network Dependency

The system depends on external weather APIs to obtain outdoor humidity data. If the internet connection is unavailable, the system will not be able to compare indoor and outdoor conditions.

Mitigation strategy is to notify the user about API connectivity issues and continue monitoring indoor conditions locally.